# Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification

AUTHOR NAME, Institution, Country

An acquired mutex must eventually be released. An opened handle must eventually be closed, on every path. Pre- and post-conditions cannot say this: the discharge point lies in code that has not been written yet.

We present *Pledge*, a Haskell library in which such obligations are ordinary values. A `Pledge m eff a` carries, beside its result, what must have held before it ran, what it produces, and what must still hold after. One bind formula propagates all three by *quotient*: binding strips from a pending obligation whatever the continuation actually delivers. What is left over — the *residual* — names precisely what the program still owes. Preconditions need no separate machinery: they are future conditions read backwards, and the two quotients are one operation under time reversal.

The formula needs only six operations, which we instantiate four ways: regular expressions with complement, where the quotient is computed by derivative and no automaton is ever built; Presburger-guarded traces, discharged by Z3; semiring-weighted obligations, whose residual is a completion probability or a minimum discharge cost; and separation-logic heaps, where the quotient is the magic wand. One bind, four notions of obligation.

We prove the monad laws in Lean 4 from twelve algebraic axioms, and identify exactly the two side conditions under which regular expressions satisfy them — conditions the obvious way of writing an annotation quietly violates, silently reporting a violated contract for a correct program. A soundness theorem completes the picture: a discharged residual entails that every terminating trace meets its obligations. Because the underlying monad is arbitrary, the residual describes the path a program actually took and can be computed beside a real effect handler, which we demonstrate on top of the `effectful` library. `Pledge` tells you what is still owed; it does not enforce it.

CCS Concepts: • **Software and its engineering → Functional languages**; *Data types and structures*; *Software verification and validation*.

Additional Key Words and Phrases: temporal specification, monadic effects, regular expressions, Brzozowski derivatives, left-quotient, right-quotient, Presburger arithmetic, semiring weights, separation logic, resource lifecycle, Haskell, Lean 4

**ACM Reference Format:**
Author Name. 2026. Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification. *Proc. ACM Program. Lang.* 1, HASKELL (August 2026), 28 pages.

## 1 Introduction

Pre- and post-conditions are the workhorse of modular program specification. A pre-condition constrains the state in which an operation may be invoked; a post-condition describes what holds immediately after it returns. Yet many correctness properties span a *sequence* of operations, with arbitrary computation in between.

Author's Contact Information: Author Name, Institution, City, Country, author@institution.edu.


ACM 2475-1421/2026/8-ART
https://doi.org/

Consider three scenarios from the Haskell ecosystem: *(i) Mutex discipline.* An `acquire(m)` must eventually be followed by `release(m)` on every code path. A post-condition records the acquired state but cannot require a future `release`. *(ii) Database savepoints.* A `beginTx` must eventually be committed or rolled back; a pre/post pair on the call cannot carry this obligation forward through intermediate code. *(iii) File handles.* Opening a file in read-mode must be followed by a close, and no write operation may be issued until a write-mode open has been posted.

*Existing remedies and their limits.* The `bracket` function lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. `ResourceT` [Snoyman 2011] threads cleanup through every bind but models only the cleanup half of an obligation, discharging it uniformly at scope exit. Linear Haskell [Bernardy et al. 2018] enforces "consume exactly once" but cannot express branching discharge or quantitative obligations. Parameterised and indexed monads [Atkey 2009] encode protocol states at the type level but require non-standard syntax and are confined to a single domain.

*The gap.* Future conditions themselves are not new. Song et al. [2025] introduce them as a first-class extension to pre/post contracts and give a logical semantics; Song et al. [2024] encode temporal properties as regular expressions and use derivatives to drive automated program repair in the ProveNFix tool. Neither, however, treats an obligation as a *value* that a program can carry, combine, and hand on. The first works at the level of a logical system, with no implementation; the second embeds its reasoning inside a whole-program analysis that a programmer cannot invoke compositionally, and that is specific to traces. What is missing is an account of how an obligation propagates through effectful code one bind at a time — and, with it, a library a Haskell programmer can actually use.

*Why a library, and not a type system or a model checker.* Temporal properties have been attacked from many directions: encoded in indexed or graded types, discharged by a model checker, or proved in a theorem prover. Each buys stronger guarantees at a cost. Type-level encodings force the protocol into the type of every intermediate computation, so a change to the specification perturbs type signatures throughout the program, and each domain (traces, heap ownership, cost budgets) needs its own encoding. Model checkers and provers work on a model of the program rather than the program, and demand that the property be fixed before the code is written. Making the obligation an ordinary Haskell *value* costs the static guarantee, but buys three things those approaches do not offer together: obligations can be computed — built from runtime data, passed to functions, stored in data structures; a single bind formula serves every domain, so a new obligation language is a new **`Composable`** instance rather than a new encoding; and the unmet remainder is itself an inspectable value, so a failure *names* what is still owed instead of reporting that a proof did not go through.

*What Pledge does, and what it does not.* A **`Pledge`** `m eff a` value is a computation in the underlying monad `m` that additionally produces three annotation fields $(pre, post, fut) :: eff$, and composing such values propagates those annotations algebraically alongside whatever `m` does. Two consequences are worth stating before the details.

First, the annotations are *path-sensitive*. Because `m` is a monad, bind must run the first action before it can apply the continuation to the result, so the annotations that come out describe the one path the program actually took — not an over-approximation of every path it might have taken. A specification therefore need not stay detached from the code it describes: §3.4 pairs a **`Pledge`** term with a real effect handler so that a single **`do`** block drives both, and lets annotations mention the runtime values the handler produces.

Second, nothing intervenes. The residual is a value to be inspected; a program that fails to discharge an obligation still runs to completion, and **`Pledge`** names the omission rather than preventing it. Promoting the residual to a runtime guard — checking it after each bind and failing on $\varnothing$ — is a short step from what is already here, but it is future work (§11) and we claim nothing about it.

*Our approach.* A value of type **`Pledge`** `m eff a` carries three annotations: a precondition *pre* :: *eff* (what must have held before), a postcondition *post* :: *eff* (what this action produces), and a future condition *fut* :: *eff* (what must hold afterwards). Monadic bind propagates these through two quotient operations on `eff`:

$$pre(p \gg\!= q) \;=\; pre(p) \sqcap (pre(q) \;/\; post(p)) \tag{1}$$
$$post(p \gg\!= q) \;=\; post(p) \cdot post(q) \tag{2}$$
$$fut(p \gg\!= q) \;=\; (fut(p) \setminus post(q)) \sqcap fut(q) \tag{3}$$

Here $\setminus$ is the *left quotient* (strips from a pending obligation whatever the continuation actually produced) and $/$ is the *right quotient* (checks whether a produced postcondition satisfies the continuation's precondition). Both are quotients of one specification by another, not by a single event: $r \setminus s$ is the residual of $r$ after *any* behaviour described by $s$. A fully-discharged residual — $pre = \neg\varnothing$ and $fut = \neg\varnothing$ — is the certificate that the composed program is contract-correct; anything else names what is still owed.

*One operation, two directions.* The two quotients are not independent machinery. A future condition is an obligation on the trace still to come; a precondition is an obligation on the trace already past; and reversing time exchanges them. Concretely, in the RE instance the right quotient *is* the left quotient conjugated by reversal,

$$r \;/\; s \;=\; \mathrm{rev}\big(\mathrm{rev}(r) \setminus \mathrm{rev}(s)\big),$$

the same regular expression $\Sigma^* \cdot e \cdot \Sigma^*$ serves as "$e$ happened earlier" in a *pre* slot and "$e$ must happen later" in a *fut* slot, and the four right-quotient axioms of §4 are the four left-quotient axioms with the direction of composition flipped. This symmetry is what makes *pre* cheap — it reuses the machinery *fut* already needs — and it also explains the one place the analogy stops: monadic bind is *oriented*, composing forward, so the backward-facing field cannot satisfy the monad laws in the same orientation. We return to this in §4.

The **`Composable`** typeclass abstracts six operations: $(\cdot, \sqcap, \varepsilon, \top, \setminus, /)$. It is closed under products and under pointwise lifting through any applicative functor, so instances compose; we give four base instances:

- *(Trace protocol,* RE*)* Extended regular expressions over a typed event alphabet, with complement as a first-class constructor — possible because $\partial_a(\neg r) = \neg\,\partial_a(r)$, so no automaton is ever built. Quotients are computed by Antimirov partial derivatives, which keep intermediate residuals small; the right quotient is obtained by reversal. $\mathrm{LTL}_f$ operators embed directly.
- *(Arithmetic bounds,* GRE*)* Each RE is paired with a Presburger arithmetic predicate over heap addresses, checked by Z3 via SBV. One annotation simultaneously constrains a protocol ordering and an arithmetic bound — neither of which the other can express.
- *(Quantitative obligation,* WRE*)* Boolean membership is generalised to a semiring weight, so the residual carries a number rather than a verdict: under the probability semiring it is the likelihood that the outstanding obligations are met, and under the tropical semiring the minimum cost of discharging them ($\infty$ when no path does).

• *(Heap ownership,* SL*)* Separation-logic heap predicates with separating conjunction as sequential composition and the magic wand as quotient. The residual is recorded symbolically; unlike the other three instances it is not yet discharged by a solver (§11).

*Contributions.*

(1) **The `Pledge` monad transformer** (§3), which makes future conditions first-class Haskell values. A single bind formula propagates all three annotations through arbitrary intervening computation, and the residual it leaves behind is itself an ordinary value the programmer can inspect.
(2) **The `Composable` algebra** (§3.1): six operations that suffice to state that formula, closed under products and pointwise lifting. We give four instances — traces (§5), arithmetic-guarded traces (§6), semiring-weighted obligations (§7), and heap ownership (§8) — differing in what an obligation *is*, but sharing the propagation rule entirely.
(3) **Mechanised metatheory, and its exact boundary** (§4). We identify twelve algebraic axioms sufficient for the three monad laws on the (*ret*, *post*, *fut*) triple and prove the laws `sorry`-free in Lean 4. Mechanising the RE instance against those axioms then *delimits* them: two hold only when the divisor denotes a non-empty language, and two — the ones associativity depends on — hold in one direction only, so associativity in general is an inclusion rather than an equality. We give the counterexample, and then the sufficient condition that restores equality: every postcondition must denote a single word. This is not a restriction in practice, because it is exactly what primitives produce and it is preserved by Equation (2).
(4) **Two side conditions an annotation must satisfy** (§4, §5.1), neither of which is apparent from the types. Alongside the single-word condition on *post*, a precondition must describe the *whole* preceding trace rather than the immediately preceding event: a *pre* of the latter kind does not survive composition, collapsing to ∅ and reporting a violated contract for a program that is in fact correct. We state both conditions, show what goes wrong without them, and give the combinators that maintain them.
(5) **A soundness theorem** for the RE instance (§5.1): a fully discharged residual entails that every terminating trace of the program satisfies every obligation its primitives name.
(6) **Evidence that the design works on real code** (§3.4, §9). We pair a `Pledge` specification with a real effect handler built on the `effectful` library, so that one `do` block drives specification and execution together. Across the case studies every correct program discharges to *pre* = *fut* = ¬∅, and every incorrect one leaves a residual naming exactly what it got wrong — an unreleased lock, an unmatched `beginTx`, a leaked handle, an arithmetic overflow, a quantitative shortfall, an ownership leak.

## 2 Background

### 2.1 Extended Regular Expressions over Typed Events

Events are the atomic units of observation. An `Event t` carries a name and a typed payload:

```
data Term    = Str String | Num Int | List [Term]
data Event t = Atom String t   -- concrete event, e.g. Atom "acquire" (Num 1)
             | Wildcard        -- matches any single event
```

The type parameter `t` allows the payload to be any type, including runtime values such as file handles or thread identifiers. The specification language is an extended regular expression type:

```
data RE t = Bot | Epsilon | Single (Event t)
          | Seq (RE t) (RE t) | Or (RE t) (RE t)
          | And (RE t) (RE t) | Star (RE t) | Not (RE t)
```

The constructors are standard: $\varnothing$ (**Bot**), $\varepsilon$ (**Epsilon**), single-event match (**Single**), concatenation (Seq), union (Or), intersection (And), Kleene star (**Star**), and complement (Not). **Wildcard** matches any single event. We include And as a primitive so its derivative is computed directly without three extra Not nodes.

Five derived forms appear throughout the paper:

$$\begin{aligned}
\texttt{anything} &\triangleq \neg\varnothing && \text{(universal language)}\\
\texttt{finally}(e) &\triangleq \texttt{anything} \cdot e \cdot \texttt{anything} && (e \text{ must eventually occur})\\
\texttt{globally}(e) &\triangleq e^* && (e \text{ at every step})\\
\texttt{never}(e) &\triangleq \neg\texttt{finally}(e) && (e \text{ must never occur})\\
\texttt{noUntil}(e, g) &\triangleq \neg\big((\Sigma \setminus \{g\})^* \cdot e \cdot \texttt{anything}\big) && (e \text{ must not occur before } g)
\end{aligned}$$

previously($e$) is an alias for finally($e$) used in pre slots to assert that $e$ occurred somewhere in the preceding trace; the underlying RE is identical, but the name signals intent.

### 2.2 Algebraic Complement and Brzozowski Derivatives

The Brzozowski derivative [Brzozowski 1964] $\partial_a(r)$ computes the language of continuations after reading event $a$. The key law for complement is:

$$\partial_a(\neg r) \;=\; \neg\big(\partial_a(r)\big) \tag{4}$$

Together with the De Morgan laws and double-negation elimination, this lets complement be propagated purely algebraically, without building a DFA. The nullability function $\nu(r)$ (= True iff $\varepsilon \in L(r)$) satisfies $\nu(\neg r) = \neg\,\nu(r)$.

```
nullable :: RE t → Bool
nullable (Not r)     = not (nullable r)
nullable (Star _)    = True
nullable (Seq r1 r2) = nullable r1 && nullable r2
nullable (Or  r1 r2) = nullable r1 || nullable r2
nullable (And r1 r2) = nullable r1 && nullable r2
nullable Epsilon     = True
nullable _           = False

derivative :: Eq t ⇒ Event t → RE t → RE t
derivative e (Not r)      = Not (derivative e r)   -- da(Not r) = Not(da(r))
derivative e (Single p)   = if subsumesEvent e p then Epsilon else Bot
derivative e (Seq r1 r2)
    | nullable r1 = Or (Seq (derivative e r1) r2) (derivative e r2)
    | otherwise   = Seq (derivative e r1) r2
derivative e (Or  r1 r2)  = Or  (derivative e r1) (derivative e r2)
derivative e (And r1 r2)  = And (derivative e r1) (derivative e r2)
derivative e (Star r)     = Seq (derivative e r) (Star r)
derivative _ _            = Bot
```

The firstWith function returns the events from a supplied finite alphabet $\Sigma_0$ that can begin a word in $L(r)$. Complement requires care: naively returning $\mathit{first}(\neg r) = \mathit{first}(r)$ is unsound. The correct characterisation is:

$$e \in \mathit{first}(\neg r, \Sigma_0) \iff e \in \Sigma_0 \;\wedge\; \lceil \partial_e(r) \rceil \neq \neg\varnothing \tag{5}$$

That is, $e$ can open a word in $L(\neg r)$ iff, after consuming $e$, the residual $\partial_e(r)$ does not cover the entire language.

### 2.3 Embedding LTL$_f$

LTL over finite traces [De Giacomo and Vardi 2013] is the natural logic for the obligations of §1: a mutex is released, or a handle closed, within the run, not somewhere along an infinite one. Its standard translation goes through an automaton, but because complement is a first-class constructor here, the LTL$_f$ operators translate directly into RE with no automaton construction. Table 1 gives the full mapping.

Table 1. LTL$_f$ to RE translation. $\Sigma^1 \triangleq$ `Single Wildcard`, $\neg\varnothing \triangleq \neg\varnothing$.

| LTL formula | RE $[\![\cdot]\!]$ | |
|---|---|---|
| $\top$ | $\neg\varnothing$ | |
| $\bot$ | $\varnothing$ | |
| $e$ | $\{e\}$ | |
| $\neg\phi$ | $\neg[\![\phi]\!]$ | |
| $\phi \wedge \psi$ | $[\![\phi]\!] \cap [\![\psi]\!]$ | |
| $\phi \vee \psi$ | $[\![\phi]\!] \cup [\![\psi]\!]$ | |
| $X\,\phi$ | $\Sigma^1 \cdot [\![\phi]\!]$ | |
| $F\,\phi$ | $\neg\varnothing \cdot [\![\phi]\!]$ | |
| $G\,\phi$ | $\neg(\neg\varnothing \cdot \neg[\![\phi]\!])$ | |
| $\phi\,U\,\psi$ | $\mathit{step}(\phi)^* \cdot [\![\psi]\!]$ | ($\phi$ propositional) |

## 3 The Pledge Monad

### 3.1 The `Composable` Class

We abstract the specification algebra via a type class with six operations:

```
class Composable a where
  concatenation :: a → a → a   -- (·), identity: empty
  conjunction   :: a → a → a   -- (⊓), identity: universe
  empty         :: a           -- identity for (·)
  universe      :: a           -- identity for (⊓)
  leftQuotient  :: a → a → a   -- dividend \ divisor
  rightQuotient :: a → a → a   -- dividend / divisor
```

The library provides Unicode infix aliases: `(·) = concatenation`, `(⊓) = conjunction`, `a \ b = leftQuotient b a`, and `p / q = rightQuotient q p`. The swapped arguments in the infix definitions are intentional: *fut* \ *post* means "left-quotient of *fut* by *post*," i.e., strip *post* from the left of *fut*. Similarly, *pre* / *post* means "right-quotient of *pre* by *post*," i.e., strip *post* from the right of *pre*.

*Algebraic roles.* `concatenation` sequences effects; `conjunction` combines simultaneous constraints; `empty` and `universe` appear in `pure`. The two quotient operations implement *obligation discharge*. Intuitively:

- *fut* \ *post*: given that the continuation has posted *post*, what of the original future obligation *fut* remains unmet from the left?
- *pre* / *post*: given that the current action has produced *post*, what portion of the next action's precondition *pre* is not yet satisfied from the right?

Two base cases anchor both quotients: $r \setminus \varepsilon = r$ (nothing observed, obligation unchanged) and $r \setminus \top = \top$ (universal postcondition covers anything). The symmetric laws hold for $/$.

*Component-wise lifting.* The instance for pairs lifts `Composable` component-wise, enabling multiple independent specification dimensions to be combined:

```
instance (Composable a, Composable b) ⇒ Composable (a, b) where
  concatenation (a1,b1) (a2,b2) = (concatenation a1 a2, concatenation b1 b2)
  leftQuotient  (a1,b1) (a2,b2) = (leftQuotient  a1 a2, leftQuotient  b1 b2)
  -- etc.
```

### 3.2 The Pledge Monad Transformer

```
newtype Pledge m eff a = Pledge { runPledge :: m (a, eff, eff, eff) }
```

A value of type `Pledge m eff a` is a computation in the underlying monad `m` that produces a 4-tuple $(a, pre, post, fut)$: the return value and three annotation fields. When `m = IO`, the annotations can be built from actual runtime values (e.g., a freshly allocated file handle), because the 4-tuple is produced by the `IO` action itself.

Two combinators support interoperability:

```
-- lift a plain m-action with trivial annotations
liftPledge :: (Composable eff, Applicative m) ⇒ m a → Pledge m eff a
liftPledge ma = Pledge $ fmap (, universe, empty, universe) ma

-- run once and collect all four components
inspect :: Functor m ⇒ Pledge m eff a → m (PledgeResult eff a)
inspect (Pledge ma) = fmap (\(a,pre,post,fut) → PledgeResult a pre post fut) ma
```

`liftPledge` embeds any `m`-action as a `Pledge` with no precondition, no postcondition, and no future obligation. `inspect` runs the action exactly once and packages the result; callers should prefer `inspect` over the individual field accessors (`getPre`, `getPost`, `getFut`) when `m` has side effects.

### 3.3 Monad Instance

`pure` introduces a value with no effect and no obligation:

```
pure x = Pledge $ pure (x, universe, empty, universe)
```

Bind sequences two computations and propagates annotations according to Equations (1)–(3):

```
Pledge ma >>= g = Pledge $ do
    (a, preA, postA, futA) <- ma
    (b, preB, postB, futB) <- runPledge (g a)
    return (b,  preA ⊓ (preB / postA),
                postA · postB,
               (futA \ postB) ⊓ futB)
```

*Future condition.* The expression $fut(A) \setminus post(B)$ computes the residual of $A$'s future obligation after observing $B$'s postcondition: events discharged by $post(B)$ are removed, and the remainder is conjoined with $B$'s own future obligation. When the result normalises to $\top$, all future obligations are discharged.

*Precondition.* The expression $pre(B) \mathbin{/} post(A)$ is the *residual precondition*: the part of $B$'s precondition not yet satisfied by $A$'s postcondition. If $post(A)$ does not cover $pre(B)$, the residual is $\varnothing$, flagging the violation. If it fully covers $pre(B)$, the residual is $\varepsilon$ and the combined precondition reduces to $pre(A)$, matching the standard Hoare sequencing rule. As discussed in §4, the `pre` field follows a Hoare-rule discipline rather than strict monad-law equality; the three monad laws are proved for the $(ret, post, fut)$ triple.

### 3.4 How the Specification Meets Real Effects

Because annotations live inside the underlying `m`, the **`Pledge`** monad transformer integrates naturally with real effectful code. Two usage patterns are common.

*Pure annotations (pure composition).* When `m = IO` and the specification primitives use **`Pledge`** `\$ return (val, pre, post, fut)`, annotations are pure Haskell values and the `IO` action does nothing. Composing such primitives with $\gg\!=$ builds a specification term; the programmer inspects `pre` and `fut` of the result to check contract-correctness, then separately executes the real effectful code.

*Live-handle annotations.* When `m = IO` and the underlying action is real, the annotations can reference actual runtime values. For example:

```
openFile :: FilePath → IO.IOMode → Pledge IO (RE IO.Handle) IO.Handle
openFile fn mode = Pledge $ do
    h <- IO.openFile fn mode            -- actual IO
    return (h, universe,
            Single (Atom "open" h),     -- post uses the real handle h
            finally (Atom "close" h))   -- fut requires closing h specifically
```

Here `h :: IO.Handle` is the actual file handle produced at runtime, embedded directly into the `RE IO.Handle` annotations. The left-quotient machinery then propagates this concrete handle through subsequent binds, so a residual `finally(close h)` names the specific unclosed handle. This is the mechanism by which dynamic runtime values appear in residual obligations without requiring a data-dependent `a → eff` type for the future field.

## 4 Monad Laws and Their Side Conditions

Every monad must satisfy three equational laws. This section proves that **`Pledge`** does, and is equally precise about what that statement excludes. There are two exclusions, and neither is visible in the types. The first is structural and expected: the *pre* field is oriented against the direction of bind, and is carried beside the lawful triple rather than inside it (§4.1). The second is a genuine side condition on how annotations are written, which we isolate in §4.4–§4.6: the obvious way to write a postcondition or a precondition can break the laws, and does so silently — reporting a violated contract for a program that is in fact correct.

### 4.1 Why *pre* sits outside the triple

Recall from §1 that *pre* and *fut* are the same notion pointing in opposite directions, and $/$ is $\backslash$ conjugated by reversal. The monad laws break that symmetry, because bind does not respect it: $p \gg\!= q$ runs $p$ and *then* $q$, so every law equates rearrangements of a forward-composed sequence. A field that accumulates forwards can satisfy such laws; a field that accumulates backwards cannot be expected to, and *pre* does not.

Concretely, left identity requires the annotations of `pure` $x \gg\!= f$ to equal those of $f\ x$. For *fut* this holds because `pure` contributes $\top$ on the left, and $\top$ is the unit that $\backslash$ preserves (axiom L2

below). For *pre* the corresponding step needs $\top \,/\, post(f\,x) = \top$, which is axiom R2 — the *mirror* of L2, and true for exactly the same reason. So *pre* is not lawless; it satisfies the mirror-image laws. What it cannot do is satisfy the forward-oriented laws simultaneously with *fut* under a single equality, because a detected precondition violation must stay visible under further binding rather than being normalised away.

We therefore prove the three laws for the triple $(ret, post, fut)$ and carry *pre* alongside, in the manner of a writer-monad log: inspected, propagated by Equation (1), and not required to satisfy the monad laws on its own. All four fields *are* covered by the axioms below; the asymmetry is in which equations we then ask them to satisfy, not in how they are treated algebraically.

### 4.2 Twelve axioms

The laws hold assuming the twelve axioms in Table 2, which we state abstractly over the **`Composable`** algebra. The reversal duality is visible in the table: R1–R4 are L1–L4 with the order of composition reversed, and nothing else.

Table 2. The twelve **`Composable`** axioms required to prove the monad laws for the $(ret, post, fut)$ triple.

| Label | Axiom | Used for |
|---|---|---|
| S1 | $\varepsilon \cdot a = a$ | Law 1, post |
| S2 | $a \cdot \varepsilon = a$ | Law 2, post |
| S3 | $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ | Law 3, post |
| C1 | $a \sqcap b = b \sqcap a$ | Law 2, pre / fut |
| C2 | $(a \sqcap b) \sqcap c = a \sqcap (b \sqcap c)$ | Law 3, pre / fut |
| C3 | $\top \sqcap a = a$ | Laws 1, 2 |
| L1 | $r \setminus \varepsilon = r$ | Law 2, fut |
| L2 | $\top \setminus r = \top$ | Law 1, fut |
| L3 | $r \setminus (a \cdot b) = (r \setminus a) \setminus b$ | Law 3, fut |
| L4 | $(a \sqcap b) \setminus c = (a \setminus c) \sqcap (b \setminus c)$ | Law 3, fut |
| R1 | $a \,/\, \varepsilon = a$ | Law 1, pre |
| R2 | $\top \,/\, r = \top$ | Law 2, pre |
| R3 | $(a \,/\, b) \,/\, c = a \,/\, (c \cdot b)$ | Law 3, pre |
| R4 | $(a \sqcap b) \,/\, c = (a \,/\, c) \sqcap (b \,/\, c)$ | Law 3, pre |

### 4.3 The three laws

We verify all three laws using these axioms. In each case we show that both sides of the law produce equal $(ret, post, fut)$ triples. It is worth noting which axioms each law consumes, because §4.4 shows that not all twelve are unconditionally available: left identity uses S1, C3, L2, R1; right identity uses S2, C1, C3, L1, R2; and associativity uses S3, C2, L3, L4, R3, R4.

*Left identity:* `pure x >>= f = f x`. Let $p =$ `pure` $x = (x, \top, \varepsilon, \top)$ and $q = f\,x$. By Equations (1)–(3):

$$
\begin{aligned}
ret &: ret(q).\checkmark \\
post &: \varepsilon \cdot post(q) = post(q). \quad \text{(S1)}\checkmark \\
fut &: (\top \setminus post(q)) \sqcap fut(q) = \top \sqcap fut(q) = fut(q). \quad \text{(L2, C3)}\checkmark
\end{aligned}
$$

For *pre*: $pre(q) \,/\, \varepsilon = pre(q)$ (R1), so $pre = \top \sqcap pre(q) = pre(q)$ (C3). ✓

*Right identity:* `e >>= pure = e`. Let $q = \mathsf{pure}(ret(e)) = (ret(e), \top, \varepsilon, \top)$.

$$\begin{aligned} ret &: ret(e).\checkmark \\ post &: post(e) \cdot \varepsilon = post(e). \quad \text{(S2)}\checkmark \\ fut &: (fut(e) \setminus \varepsilon) \sqcap \top = fut(e) \sqcap \top = \top \sqcap fut(e) = fut(e). \quad \text{(L1, C1, C3)}\checkmark \end{aligned}$$

*Associativity.* Let $r_1 = ret(e)$, $r_2 = ret(f\, r_1)$. Both sides yield $ret(g\, r_2)$ and $post(e) \cdot post(f\, r_1) \cdot post(g\, r_2)$ (by S3). For the future, unfolding Equation (3) on the LHS and applying L4 then L3 gives:

$$\big((fut(e) \setminus post(f\, r_1)) \setminus post(g\, r_2)\big) \sqcap (fut(f\, r_1) \setminus post(g\, r_2)) \sqcap fut(g\, r_2)$$

which equals the RHS by C2 and L3. ✓ For *pre*, applying R4 then R3 gives the same result on both sides by C2. ✓

### 4.4 What the RE instance actually satisfies

The proofs above are relative to the twelve axioms. Mechanising the RE instance against them does not merely confirm the axioms — it *delimits* them. Eight hold outright; four do not, and Table 3 records which.

Table 3. Status of the twelve axioms for the RE instance, as mechanised in `Formalization/REComposableLaws.lean`.

| Axioms | Status | Condition |
|---|---|---|
| S1–S3, C1–C3 | hold | — |
| L1, L3, R1, R3 | hold | — |
| L2, R2 | hold conditionally | $L(\text{divisor}) \neq \varnothing$ |
| L4, R4 | one inclusion only | $\subseteq$ holds; $\supseteq$ fails |

*Why L2 and R2 need a non-empty divisor.* $\top \setminus r$ collects every suffix left over after consuming some word of $L(r)$. If $L(r) = \varnothing$ there is no such word and the quotient is $\varnothing$, not $\top$. The condition is therefore exactly $L(r) \neq \varnothing$: a postcondition denoting the empty language would assert that the action produced an impossible trace.

*Why L4 and R4 hold in only one direction.* Written out, L4 asks $(a \sqcap b) \setminus c = (a \setminus c) \sqcap (b \setminus c)$. The left-hand side demands *one* word $u \in L(c)$ that works for both conjuncts; the right-hand side lets each conjunct choose its own. Whenever the two need different witnesses, the sides differ. Taking $L(c) = \{a, b\}$, $L(a) = \{a\}$ and $L(b) = \{b\}$: on the left, $L(a) \cap L(b) = \varnothing$, so the quotient is $\varnothing$; on the right, the first conjunct picks $u = a$ and the second picks $u = b$, each leaving $\varepsilon$, so the meet is $\{\varepsilon\}$. Only $\subseteq$ survives.

*Consequence: associativity is an inclusion.* Associativity consumes L4 and R4, so in general it holds only as

$$fut\big((p \gg\!= f) \gg\!= g\big) \;\subseteq\; fut\big(p \gg\!= (\lambda x.\, f\, x \gg\!= g)\big),$$

and symmetrically for *pre*. The left-nested bracketing yields the *stronger* obligation. This is not a vacuous gap: over a pool of nine representative annotations in each of the five positions, the two bracketings differ in 902 of 59,049 combinations — and in every one of them the inclusion above is strict in the direction stated.

### 4.5 Single-word postconditions restore equality

The gap closes under a condition that costs nothing in practice.

Lemma 4.1 (Unique witness). *If $L(c)$ is a single word $\{u\}$, then L4 and R4 hold with equality, and L2 and R2 hold.*

Proof. $\{u\}$ is non-empty, giving L2 and R2. For L4, $(a \sqcap b) \setminus \{u\} = \{w \mid uw \in L(a) \cap L(b)\} = \{w \mid uw \in L(a)\} \cap \{w \mid uw \in L(b)\} = (a \setminus \{u\}) \sqcap (b \setminus \{u\})$; the choice of witness is forced, so the two sides cannot diverge. R4 is the mirror image. □

Theorem 4.2 (Laws for single-word postconditions). *If every primitive's post denotes a single word, then all twelve axioms hold for the* RE *instance and the three monad laws hold with equality.*

Proof. Every $\setminus$ and $/$ in Equations (1) and (3) has a *post* field as its divisor, so Lemma 4.1 applies at every use. The remaining eight axioms hold unconditionally. □

The hypothesis is not a restriction, because it is what the library produces anyway and is preserved by composition. A primitive emits exactly one event; `pure` and **`liftPledge`** emit $\varepsilon$; and by Equation (2) the postcondition of a composite is $post(p) \cdot post(q)$, a concatenation of two single-word languages and hence itself a single word. The class is therefore closed under bind, and every *post* arising in a **`Pledge`** program built from annotated primitives satisfies it by construction. What the condition rules out is a primitive that advertises a *choice* of postconditions — `Or`, **`Star`** or a complement in a *post* slot. Such an annotation is expressible, and silently costs associativity; §11 returns to enforcing the restriction in the API.

### 4.6 Preconditions must describe the whole past

A second side condition governs *pre*, and violating it is the more insidious failure of the two, because it produces a confident report of a contract violation for a correct program.

The natural way to annotate `release` is "an `acquire` must have happened", and the natural way to write that is $pre = \{\texttt{acquire}\}$ — the singleton language. That is wrong, and not for the reason one expects. It says the acquire must be the *immediately preceding* event, and such a condition does not survive composition. Unfolding Equation (1) for $p \gg\!= q$ with $pre(q) = \{e\}$ and $post(p) = \{e\}$:

$$pre(q) \,/\, post(p) \;=\; \{w \mid w \cdot e \in \{e\}\} \;=\; \{\varepsilon\},$$

so $pre(p \gg\!= q) = pre(p) \sqcap \{\varepsilon\}$, which is $\varnothing$ whenever $pre(p)$ does not accept the empty trace. The obligation was discharged, and the residual nevertheless reports a violation.

The fix is to require preconditions to be closed under extension on the left — properties of the entire preceding trace, not of its last event.

*Definition 4.3 (Past-closed).* A language $P$ is *past-closed* if $P = \Sigma^* \cdot P$: whenever $w \in P$ and $u \in \Sigma^*$, also $uw \in P$.

Proposition 4.4 (Closure). *Past-closed languages contain $\top$ and are closed under $\sqcap$ and under $/$ by an arbitrary divisor. Consequently, if every primitive's pre is past-closed, so is the pre of every program composed from them.*

Proof. $\top = \Sigma^*$ is past-closed. For $\sqcap$: $\Sigma^*(A \cap B) \subseteq \Sigma^*A \cap \Sigma^*B = A \cap B$, and the reverse inclusion takes $u = \varepsilon$. For $/$: let $w \in P \,/\, s$, so $wu \in P$ for some $u \in L(s)$; for any $v$, past-closure of $P$ gives $vwu \in P$, hence $vw \in P \,/\, s$. The final claim follows by induction on the structure of the program, since Equation (1) builds *pre* from $\sqcap$ and $/$ alone. □

The $\texttt{previously}(e) = \Sigma^* \cdot e \cdot \Sigma^*$ combinator of §2.3 is past-closed, and is the intended way to write a precondition; the singleton $\{e\}$ is not. With it, the collapse above does not occur: $\texttt{previously}(e) \,/\, \{e\} = \Sigma^*$, the obligation is discharged to $\top$, and the enclosing $\sqcap$ leaves $pre(p)$ untouched. Proposition 4.4 guarantees this is stable under further composition.

Neither this condition nor Theorem 4.2 is enforced by the type `eff`, and both are easy to violate by writing the annotation that first comes to mind. We regard stating them as part of the contribution: an instance author must honour them, and §11 discusses making them unwriteable rather than merely documented.

### 4.7 Mechanisation and validation

The three law proofs are mechanised in Lean 4 in `Formalization/PledgeMonadLaws.lean`, using an abstract `Composable` structure parametrised over the twelve axioms of Table 2. The file carries no `sorry`; all proofs are term-mode derivations from the stated axioms, and depend on Mathlib [The Mathlib Community 2024] only for basic list and set lemmas. A companion file, `Formalization/REComposableLaws.lean`, develops the RE instance at the level of languages: it proves the eight unconditional axioms, proves L2 and R2 under the non-emptiness hypothesis, proves the $\subseteq$ direction of L4 and R4, and exhibits the counterexample to the converse. Derivative correctness and `nullable` correctness are proved there as well; the correspondence between the language-level quotient and its implementation by derivatives is assumed, as is standard.

We complement the mechanisation with exhaustive testing over bounded pools, which is what first exposed the boundary. Quotients are checked against their set-theoretic definition $w \in r \setminus s \iff \exists u \in L(s).\, uw \in L(r)$ over all pairs from a pool of 90 regular expressions and all words up to length two — 56,700 checks for each quotient, with no discrepancy. The axioms are then checked over the same pool, reproducing Table 3 exactly and independently of the Lean development. Finally, restricting postconditions to single words as Theorem 4.2 requires, and letting futures and preconditions range over RE terms including complement, star and intersection, associativity holds in all 43,200 combinations tested — against the 902 failures found without the restriction.

## 5 The RE Instance

The `Composable (RE t)` instance maps the algebra operations onto RE constructors and implements the quotients via Brzozowski derivatives. The left-quotient $r_2 \setminus r_1$ — "what of $r_2$ remains after $r_1$ is consumed from the left" — is computed by Antimirov partial derivatives on $r_2$, which keep intermediate terms smaller than the Brzozowski derivative while producing the same language.

```
instance Eq t ⇒ Composable (RE t) where
  concatenation = Seq
  conjunction   = And
  empty         = Epsilon
  universe      = Not Bot        -- = Sigma*
  leftQuotient  = reLeftQuotient
  rightQuotient = reRightQuotient
```

The right-quotient is computed via reversal: $r_2 / r_1 = rev(rev(r_2) \setminus rev(r_1))$, where $rev(r)$ reverses the sequences accepted by $r$. A normaliser applies the laws in Table 4 to simplify RE terms, exploiting the complement constructor to push `Not` inward via De Morgan's laws.

*Termination.* The left-quotient recursion terminates because the set of distinct normalised derivatives of any regular expression is finite — a classical result of Brzozowski [Brzozowski 1964],

Table 4. Normalisation laws applied by `normalize`. All rules are applied recursively bottom-up.

| Law | Constructor |
|---|---|
| $\varepsilon \cdot r = r, r \cdot \varepsilon = r$ | `Seq` |
| $\varnothing \vee r = r, r \vee \neg\varnothing = \neg\varnothing$ | `Or` |
| $\varnothing \wedge r = \varnothing, r \wedge \neg\varnothing = r$ | `And` |
| $\neg\neg r = r$ | `Not` |
| $\neg(r_1 \vee r_2) = \neg r_1 \wedge \neg r_2$ | `Not` |
| $\neg(r_1 \wedge r_2) = \neg r_1 \vee \neg r_2$ | `Not` |
| $\neg\varnothing = \neg\varnothing, \neg\neg\varnothing = \varnothing$ | `Not` |
| $\varnothing^* = \varepsilon, \varepsilon^* = \varepsilon$ | **`Star`** |

extended to complement by Owens et al. [Owens et al. 2009]. Each recursive call is made with the normalised derivative of the previous operands, so the recursion explores a path in this finite set and must eventually reach a base case.

### 5.1 Soundness

A fully-discharged residual ($\neg\varnothing$ in *fut*, $\neg\varnothing$ in *pre*) serves as a certificate of contract-correctness. We now prove that this certificate is sound for the RE instance.

For an RE $r$ and a finite trace $w$, write $w \models r$ for $w \in L(r)$ and $\partial_w(r)$ for the iterated derivative.

Lemma 5.1 (Derivative correctness). *For all $r$ and $w$: $w \models r$ iff $\nu(\partial_w(r)) = \mathtt{True}$.*

Proof. By induction on $w$. The base case is the definition of $\nu$. For $w = a \cdot w'$: $w \models r$ iff $w' \models \partial_a(r)$ (standard derivative correctness [Brzozowski 1964], extended to `Not` by Equation (4) and to `And` by the direct derivative clause); the inductive hypothesis on $w'$ and $\partial_a(r)$ gives $\nu(\partial_{w'}(\partial_a(r))) = \nu(\partial_w(r))$. □

Throughout this subsection we assume the single-word condition of Theorem 4.2: every primitive's *post* denotes one word. The assumption is what makes the statement below expressible at all. Without it, §4.4 showed that associativity holds only as an inclusion, so "the residual of $e$" is not determined by $e$ alone but by how its binds are bracketed, and there is nothing well-defined to prove sound.

Under the assumption, a program $e$ built from primitives $p_1, \ldots, p_n$ by `pure` and $\gg\!=$ emits one determinate trace: writing $post(p_i) = \{u_i\}$, Equation (2) gives $post(e) = \{u_1 u_2 \cdots u_n\}$. Call that word $w$, and write $w_{<i} = u_1 \cdots u_{i-1}$ for the trace before $p_i$ runs and $w_{>i} = u_{i+1} \cdots u_n$ for the trace after.

Theorem 5.2 (Soundness of RE-residual checking). *Let $e$ be a* `Pledge m (RE t) a` *term built from* `pure` *and $\gg\!=$ over primitives $p_1, \ldots, p_n$ whose postconditions each denote a single word. If $L(fut(e)) = \Sigma^*$ and $L(pre(e)) = \Sigma^*$, then for every $i$:*

$$w_{>i} \models fut(p_i) \qquad \text{and} \qquad w_{<i} \models pre(p_i).$$

*That is, every obligation deferred by a primitive is discharged by what follows it, and every condition demanded by a primitive is established by what precedes it.*

Proof. By induction on $n$. For $n = 0$ the program is `pure` $x$, which names no obligation and the claim is vacuous.

For the inductive step write $e = p \gg\!= f$, where $p = p_1$ and $f$ yields the term $e'$ built from $p_2, \ldots, p_n$; note $post(e') = \{u_2 \cdots u_n\} = \{w_{>1}\}$.

*Futures.* By Equation (3), $fut(e) = (fut(p) \setminus post(e')) \sqcap fut(e')$. Since $\sqcap$ is intersection and $L(fut(e)) = \Sigma^*$, both conjuncts denote $\Sigma^*$. From the first, $L(fut(p) \setminus \{w_{>1}\}) = \Sigma^*$; by definition of the quotient this says $w_{>1}\, v \models fut(p)$ for every $v$, and taking $v = \varepsilon$ gives $w_{>1} \models fut(p)$, which is the claim for $i = 1$. From the second, $L(fut(e')) = \Sigma^*$, so the induction hypothesis applies to $e'$ and gives the claim for $i \geq 2$ — noting that the trace following $p_i$ within $e'$ is exactly $w_{>i}$.

*Preconditions.* Symmetrically, Equation (1) gives $pre(e) = pre(p) \sqcap (pre(e') \,/\, post(p))$, so both conjuncts denote $\Sigma^*$. The first yields $L(pre(p)) = \Sigma^*$, so $w_{<1} = \varepsilon$ satisfies it. The second gives $L(pre(e') \,/\, \{u_1\}) = \Sigma^*$, i.e. $v\, u_1 \models pre(e')$ for every $v$; with $v = \varepsilon$ this makes $u_1$ a trace establishing $pre(e')$, and the induction hypothesis on $e'$ — whose preceding trace is extended by $u_1$ on the left — gives the claim for $i \geq 2$. □

Two remarks locate the theorem precisely.

*It is a soundness result only.* A discharged residual certifies that the obligations are met; a *non*-discharged residual does not by itself certify that they are not. The converse direction is what the past-closure condition of §4.6 protects: without it a correct program can produce $pre(e) = \varnothing$ and be reported as violating. The two side conditions are thus complementary — single-word postconditions make the certificate meaningful, past-closed preconditions make its absence meaningful.

*The implementation tests a sufficient syntactic condition.* The theorem is stated semantically, in terms of $L(\cdot) = \Sigma^*$. What `Pledge` computes is $\lceil\cdot\rceil$, a normaliser, and reports discharge when the result is syntactically $\neg\varnothing$. Normalisation is sound but incomplete for language universality, so the check can fail to recognise a discharged residual — never the reverse. The normaliser accordingly implements the identities that arise in practice, among them $\neg\varnothing \cdot r = \neg\varnothing$ for nullable $r$, without which a residual such as $\Sigma^* \cdot (a \cdot \Sigma^* \vee \varepsilon)$ would be reported as outstanding despite denoting $\Sigma^*$.

Theorem 5.2 applies to the RE instance because the proof relies on derivative correctness for regular languages. The GRE instance inherits soundness for its RE component and requires the Presburger side to be checked by a sound decision procedure (Z3), which we take as given. The SL instance does not admit an analogous soundness theorem as it stands, because its quotient records the magic wand symbolically without a semantic heap model; connecting residual predicates to a solver is identified in §11 as the natural next step.

## 6 The GuardedRE Instance

The RE instance reasons about event traces but is silent about arithmetic state. The GRE instance closes this gap: a guarded regular expression pairs a trace RE with a Presburger arithmetic predicate over heap addresses, so one annotation simultaneously enforces a protocol ordering and a numeric bound.

### 6.1 The GuardedRE Type

```
data GuardedRE a = GuardedRE PPred (RE a)
```

A state (heap, trace) satisfies `GuardedRE p r` iff heap $\models p$ and trace $\in L(r)$. The two sides are independent: $p$ is a static constraint, while $r$ advances through derivatives as events arrive.

*What the arithmetic side can and cannot say.* That independence is worth dwelling on, because it bounds the instance sharply and the bound is easy to overlook. Every operation of the algebra conjoins the two predicates and quotients only the two REs; the predicate is never advanced by an event. A GRE annotation can therefore express an *invariant* — a property of the heap that holds throughout, such as "address $a$ is live" or "$0 \leq h[a]$" — but it cannot express a property that changes as the trace proceeds.

Writing one anyway fails silently, and in the wrong direction. Suppose `init` posts $h[a] = 0$ and `dec` requires $h[a] > 0$, the natural annotation for a counter. Both predicates are about $h[a]$ at *different moments*, but the instance has no way to say so: composing the two conjoins them into $h[a] = 0 \wedge h[a] > 0$, which is unsatisfiable, so a correct program is reported as violating its contract. Meanwhile an `inc` guarded by $h[a] < max$ never contradicts $h[a] = 0$, so a run that overflows is reported as fine. The predicate must be read as a single assertion about one heap, and annotations that treat it as a sequence of assertions about successive heaps invert the verdict.

Expressing state change properly requires indexing values by trace position, which is a Presburger encoding of the trace itself rather than a predicate alongside it; we have not pursued it. The examples of §9.3 accordingly use GRE for what it supports — an invariant conjoined with a protocol ordering, where neither constraint alone suffices.

Smart constructors lift a pure RE or pure predicate:

```
fromRE    :: RE a    → GuardedRE a
fromPPred :: PPred   → GuardedRE a
```

### 6.2 The `Composable GuardedRE` Instance

```
instance Eq a ⇒ Composable (GuardedRE a) where
  concatenation (GuardedRE p1 r1) (GuardedRE p2 r2) =
      GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (Seq r1 r2))
  conjunction = conjoin
  empty    = GuardedRE PTrue Epsilon
  universe = GuardedRE PTrue (Not Bot)
  leftQuotient  (GuardedRE p1 r1) (GuardedRE p2 r2) =
      GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (reLeftQuotient r1 r2))
  rightQuotient (GuardedRE p1 r1) (GuardedRE p2 r2) =
      GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (reRightQuotient r1 r2))
```

Both quotient operations conjoin the two heap predicates: the residual must respect constraints from both operands. The RE component uses the same derivative-based quotients as the plain RE instance.

### 6.3 Presburger Predicates

Predicates are built over a linear arithmetic expression language:

```
data PExpr = Lit Int | ValAt Addr | Add PExpr PExpr | Mul Int PExpr
data PPred = PTrue | PFalse
           | PLt PExpr PExpr | PLe PExpr PExpr | PEq PExpr PExpr
           | PGt PExpr PExpr | PGe PExpr PExpr
           | PNot PPred | PAnd PPred PPred
```

`ValAt a` reads the value at heap address `a`. The normaliser `normalizePPred` flattens `PAnd` trees, removes duplicates, substitutes equalities of the form $h[a] = k$, evaluates literal comparisons, and collapses $\neg$**true** to `PFalse`. A predicate reaching the solver that is syntactically `PFalse` is rejected immediately without an SMT call. Non-trivial residual predicates are discharged by Z3 via SBV.

## 7 The WeightedRE Instance

The RE and GRE instances are Boolean: a future condition is either satisfied or violated. The WRE instance generalises membership to a weight from a semiring, enabling probabilistic and quantitative obligations.

### 7.1 The Semiring Class

```
class (Eq w, Show w) ⇒ Semiring w where
  szero :: w;  sone :: w
  sadd  :: w → w → w    -- choice / union
  smul  :: w → w → w    -- sequence / composition
```

Two concrete instances capture quantitative scenarios:

| Instance | `szero` | `sone` | `sadd` | `smul` |
|---|---|---|---|---|
| **`Prob`** (probability) | 0 | 1 | $\min(p+q,\,1)$ | $p \times q$ |
| **`Tropical`** (min-cost) | $+\infty$ | 0 | min | $+$ |

### 7.2 The `WRE` Type

```
data WRE w t
    = WBot | WEps w | WSingle w (Event t)
    | WSeq (WRE w t) (WRE w t) | WAdd (WRE w t) (WRE w t)
    | WAnd (WRE w t) (WRE w t) | WStar (WRE w t)
```

The generalised nullable function accumulates the weight assigned to the empty trace:

```
wNullable :: Semiring w ⇒ WRE w t → w
wNullable (WEps w)     = w
wNullable (WSeq r1 r2) = smul (wNullable r1) (wNullable r2)
wNullable (WAdd r1 r2) = sadd (wNullable r1) (wNullable r2)
wNullable (WAnd r1 r2) = smul (wNullable r1) (wNullable r2)
wNullable (WStar _)    = sone
wNullable _            = szero
```

Smart constructors `wFinally w e` and `wGlobally w e` define weighted analogues of `finally`$(e)$ and `globally`$(e)$:

```
wTop     = WStar (WSingle sone Wildcard)
wFinally w e = WSeq wTop (WSingle w e)
wGlobally w e = WStar (WSingle w e)
```

The **`Composable`** `(WRE w t)` instance maps **`concatenation`** to `WSeq`, **`conjunction`** to `WAnd`, and implements the quotients via weighted Brzozowski derivatives.

*Absence of weighted complement.* **`WRE`** does not include a `WNot` constructor. Boolean negation ($0 \leftrightarrow 1$) has no canonical generalisation to an arbitrary semiring, so the $\mathrm{LTL}_f$ operators that rely on complement ($G$, $U$, `never`) are unavailable. The operators `wFinally` and `wGlobally` are defined directly via `WStar` and `WSeq` without complement, covering the common quantitative patterns.

## 8 The SL Instance

The **`Composable`** typeclass is not inherently tied to trace-based specifications. We demonstrate its generality with a fourth instance over *separation-logic heap predicates*, where future conditions describe what heap ownership must hold upon completion.

### 8.1 Structural Parallel with RE

Table 5 shows the role each SL construct plays in the **`Composable`** interface.

Table 5. Structural parallel between the four `Composable` instances.

| Operation | RE | GRE | $\mathrm{WRE}_w$ | SL |
|---|---|---|---|---|
| `concatenation` | $r_1 \cdot r_2$ | $(p_1 \wedge p_2,\ r_1 \cdot r_2)$ | `WSeq` | $P * Q$ |
| `conjunction` | $r_1 \wedge r_2$ | $(p_1 \wedge p_2,\ r_1 \wedge r_2)$ | `WAnd` | $P \wedge Q$ |
| `empty` | $\varepsilon$ | $(\mathbf{true}, \varepsilon)$ | `WEps sone` | **emp** |
| `universe` | $\neg\varnothing$ | $(\mathbf{true}, \neg\varnothing)$ | `wTop` | $\top$ |
| `leftQuotient` | Brzozowski derivative | quotient (RE) + $\wedge$ (PPred) | weighted derivative | $P \mathrel{-\!\!*} Q$ |
| `rightQuotient` | reversal + left-quotient | idem | idem | $P \mathrel{-\!\!*} Q$ |

### 8.2 The SL Predicate Type

```
data SL
    = Emp              -- empty heap (identity for SepStar)
    | Top              -- any heap  (identity for Conj)
    | Pure PPred       -- pure arithmetic constraint (heap-independent)
    | Cell Addr Val    -- singleton: address a holds value v
    | SepStar SL SL    -- P * Q  separating conjunction
    | Conj    SL SL    -- P /\ Q ordinary conjunction
    | Wand    SL SL    -- P -* Q magic wand (residual)
```

### 8.3 The `Composable SL` Instance

```
instance Composable SL where
  concatenation = SepStar
  conjunction   = Conj
  empty         = Emp
  universe      = Top
  -- emp -* Q = Q  (nothing provided, obligation unchanged)
  leftQuotient Emp q = q
  -- Top is trivially universal; any Q is discharged.
  leftQuotient Top _ = Emp
  leftQuotient p   q = Wand p q   -- symbolic magic wand
  -- SepStar is commutative, so left- and right-quotients coincide.
  rightQuotient = leftQuotient
```

The two base cases mirror the RE instance: $\mathbf{emp} \mathrel{-\!\!*} Q = Q$ (providing no heap leaves the obligation unchanged) and $\top \mathrel{-\!\!*} Q = \mathbf{emp}$ (any postcondition discharges the obligation). The general case records the wand *symbolically* for later evaluation.

*Symbolic recording, not semantic discharge.* The five normalisation rules of `normalizeSL` reduce syntactically trivial wands, but the general `Wand p q` case is retained as a data structure: no heap model is consulted and no solver is invoked. The SL instance therefore records obligations and propagates them correctly through bind; it does not check whether the accumulated residual is satisfiable. Consequently, Theorem 5.2's style of guarantee does not extend to SL as it stands. Connecting residual predicates to an external solver (Z3 via `sbv` or a dedicated SL procedure such as Smallfoot) is identified in §11 as the concrete next step.

## 9 Examples

We demonstrate **Pledge** across five case studies covering all four instances. Each defines primitive operations annotated with `pre`, `post` and `fut`, composes them in the **Pledge** monad, and inspects the residual. Every future condition below is non-trivial except in the SL case, where §9.5 explains why the instance admits none.

How a residual is read varies by instance, and the four cases together are the point of the section. For RE and GRE, a *pre* and *fut* of $\neg\varnothing$ certify correctness by Theorem 5.2, $\varnothing$ marks a violation, and a residual between the two is a standing constraint on the continuation (§9.3). For WRE there is no $\neg\varnothing$ to compare against: one applies $\nu$ and reads a cost or a probability (§9.4). For SL the residual is recorded but never discharged. All figures quoted below are the output of the accompanying artefact.

### 9.1 Mutex Lifecycle (RE Instance)

An acquired mutex must eventually be released, and a release requires a prior acquire. The event alphabet is `Term`; mutex identifiers are `Num mid`.

```
acquire :: Int → Pledge IO (RE Term) ()
acquire mid = Pledge $ return
    ( ()
    , universe                                      -- pre: none
    , Single (Atom "acquire" (List [Num mid]))      -- post
    , finally (Atom "release" (List [Num mid])) )   -- fut: must release

release :: Int → Pledge IO (RE Term) ()
release mid = Pledge $ return
    ( ()
    , previously (Atom "acquire" (List [Num mid])) -- pre: acquired earlier
    , Single (Atom "release" (List [Num mid]))     -- post
    , universe )                                    -- fut: none
```

Note the precondition is `previously`, not **Single**: by §4.6 it must describe the whole preceding trace. The tempting **Single** `(Atom "acquire" ...)` — "an acquire happened" — in fact says "an acquire happened *immediately* before", and `nestedLocks` below, where `release 2` intervenes, would then report a violated contract despite being correct.
The four example programs below and their residuals illustrate the key scenarios:

```
-- Good: acquire then release
safeSection = do { acquire 1; release 1 }

-- Good: nested locks, released in reverse order
nestedLocks = do { acquire 1; acquire 2; release 2; release 1 }

-- Bad future: lock 1 never released
lockLeak = do { acquire 1; acquire 2; release 2 }

-- Bad pre: release without prior acquire
releaseWithoutAcquire = release 1
```

| Program | Residual *fut* | Residual *pre* |
|---|---|---|
| `safeSection` | ¬∅ | ¬∅ |
| `nestedLocks` | ¬∅ | ¬∅ |
| `lockLeak` | `finally(release(1))` | ¬∅ |
| `releaseWithoutAcquire` | ¬∅ | `finally(acquire(1))` |

The two correct programs discharge completely, and the two incorrect ones each leave exactly one residual naming what went wrong. For `lockLeak` the residual future names lock 1 specifically: the future condition of the first `acquire` is discharged by the `release 2` postcondition only for lock 2, so lock 1's obligation `finally(release(1))` passes through the left quotient unchanged. For `releaseWithoutAcquire` the residual *pre* is `finally(acquire(1))`, which reads as an obligation transferred to the caller: this fragment is usable only in a context that has already acquired lock 1. Composing it after `acquire 1` discharges the residual to ¬∅, which is precisely `safeSection`. `nestedLocks` is the case that exercises the propagation machinery. Two obligations are outstanding when `release 2` runs; its postcondition discharges one and leaves the other untouched, and the subsequent `release 1` discharges that. Neither a pre/post pair nor a lexically scoped `bracket` tracks this, because the obligations are released in an order the nesting does not determine.

### 9.2 Database Transactions (RE Instance)

A transaction begun with `beginTx` carries a *disjunctive* future condition requiring eventual commit or rollback.

```
beginTx :: Pledge IO (RE Term) ()
beginTx = Pledge $ return
    ( ()
    , universe
    , Single (Atom "beginTx" (List []))
    , Or (finally (Atom "commit"   (List [])))
         (finally (Atom "rollback" (List []))) )

commit :: Pledge IO (RE Term) ()
commit = Pledge $ return
    ( ()
    , previously (Atom "beginTx" (List []))  -- a tx must have begun
    , Single (Atom "commit" (List []))
    , universe )

rollback :: Pledge IO (RE Term) ()
rollback = Pledge $ return
    ( (), universe
    , Single (Atom "rollback" (List []))
    , universe )

-- Good: begin, commit
committedTx = do { beginTx; commit }

-- Good: begin, rollback
rolledBackTx = do { beginTx; rollback }
```

```
-- Bad: transaction never closed
openTx = beginTx
```

`openTx` retains the full disjunction `finally(commit)` ∨ `finally(rollback)` as its residual future. `committedTx` and `rolledBackTx` fully discharge the obligation, yielding ¬∅. The disjunction is computed by the left quotient: (`finally(commit)`∨`finally(rollback)`)\`Single(commit)` reduces to ¬∅ because `commit` advances the `finally(commit)` branch to ¬∅, and a union with ¬∅ is ¬∅.

This is the case a lexically scoped combinator handles least well. A transaction may be discharged by either of two events, in a branch chosen at run time; `bracket` must commit to one cleanup action when the scope is written. Here the obligation stays a disjunction until an event resolves it, and whichever branch runs discharges the whole.

### 9.3 Heap Liveness and Allocation Order (GRE Instance)

Manual memory management needs two constraints at once, and they are of different kinds. The *trace* constraint is that a `malloc` must precede the matching `free`, that the `free` must eventually happen, and that a second `free` must not occur before re-allocation. The *heap* constraint is that the cell is live, $h[a] > 0$. An RE cannot see liveness and a Presburger predicate cannot see ordering, so each alone admits programs the other rejects. Both are invariants of the run rather than assertions about successive states, which is what GRE supports (§6).

```
malloc :: Addr → Pledge IO (GuardedRE Term) Addr
malloc addr = Pledge $ return
    ( addr                                          -- returns the address
    , fromRE universe                               -- pre: none
    , fromRE (Single (Atom "malloc" (List [Num addr]))) -- post
    , GuardedRE (PGt (ValAt addr) (Lit 0))          -- fut: live, and
                (finally (Atom "free" (List [Num addr]))) --   free eventually
    )

free :: Addr → Pledge IO (GuardedRE Term) ()
free addr = Pledge $ return
    ( ()
    , GuardedRE (PGt (ValAt addr) (Lit 0))            -- pre: live, and
                (previously (Atom "malloc" (List [Num addr]))) -- malloc'd
    , fromRE (Single (Atom "free" (List [Num addr])))     -- post
    , fromRE (noUntil (Atom "free"   (List [Num addr]))   -- fut: no double
                      (Atom "malloc" (List [Num addr])))  --   free until realloc
    )
```

The future condition here is not ⊤: `malloc` defers a real obligation, and `free` imposes a standing one on everything that follows. Note also that `malloc` *returns* the address and the programs below bind it, so the annotation of `free` is built from a value produced at run time — the mechanism of §3.4 in miniature.

```
mallocThenFree = do { a <- malloc 1; free a }
reallocate     = do { a <- malloc 1; free a; a' <- malloc 1; free a' }

missingFree       = do { _ <- malloc 1; return () }   -- never freed
freeWithoutMalloc = free 1                            -- no allocation
doubleFree        = do { a <- malloc 1; free a; free a }
```

```
wrongFree          = do { a <- malloc 1; free (a + 1) } -- frees the wrong cell
```

| Program | *pre* | *fut* |
|---|---|---|
| `mallocThenFree` | $[h_1{>}0]\ \neg\varnothing$ | $[h_1{>}0]$ `noUntil(free`$_1$`, malloc`$_1$`)` |
| `reallocate` | $[h_1{>}0]\ \neg\varnothing$ | $[h_1{>}0]$ `noUntil(free`$_1$`, malloc`$_1$`)` |
| `missingFree` | [**true**] $\neg\varnothing$ | $[h_1{>}0]$ `finally(free`$_1$`)` |
| `freeWithoutMalloc` | $[h_1{>}0]$ `finally(malloc`$_1$`)` | [**true**] `noUntil(free`$_1$`, malloc`$_1$`)` |
| `doubleFree` | $[h_1{>}0]\ \neg\varnothing$ | $[h_1{>}0]\ \varnothing$ |
| `wrongFree` | $[h_2{>}0]$ `finally(malloc`$_2$`)` | $[h_1{>}0]$ `finally(free`$_1$`)` $\sqcap$ `noUntil(free`$_2$`, malloc`$_2$`)` |

This example shows that "discharged" admits three outcomes, not two. A residual of $\neg\varnothing$ owes nothing. A residual of $\varnothing$ is a violation: `doubleFree` reaches it because the second `free` must satisfy the `noUntil(free`$_1$`, malloc`$_1$`)` that the first one imposed, and no trace does. Between the two lies a satisfiable residual that is not $\neg\varnothing$ — a *standing constraint* on whatever runs next. `mallocThenFree` is correct and still carries `noUntil(free`$_1$`, malloc`$_1$`)`: it has discharged what it owed and, in exchange, constrains its continuation not to free cell 1 again before re-allocating. `reallocate` exercises exactly that, and the second `malloc` resets the guard.

`wrongFree` is the case where both fields carry information. Freeing cell 2 leaves cell 1's obligation `finally(free`$_1$`)` outstanding in *fut*, and simultaneously demands `finally(malloc`$_2$`)` in *pre*, since cell 2 was never allocated. One residual pair names both the leak and the unmet requirement.

### 9.4 Task Scheduling (WRE Tropical Instance)

Under the tropical (min-plus) semiring, weights represent minimum numbers of steps. Each `submit` carries a future condition requiring the task to be resolved.

```
type TSRE = WRE Tropical Term

submit :: Int → Pledge IO TSRE ()
submit taskId = Pledge $ return
    ( ()
    , wTop                                                    -- pre: none
    , WSingle (Tropical 1) (Atom "submit" (List [Num taskId]))   -- post
    , WAdd (wFinally (Tropical 1) (Atom "complete" (List [Num taskId])))
           (wFinally (Tropical 2) (Atom "abort"    (List [Num taskId]))) )

complete :: Int → Pledge IO TSRE ()
complete taskId = Pledge $ return
    ( ()
    , wPreviously (Tropical 1) (Atom "submit" (List [Num taskId]))  -- pre
    , WSingle (Tropical 1) (Atom "complete" (List [Num taskId]))    -- post
    , wTop )                                                  -- fut: none
```

The identity elements matter here and are easy to get wrong. "No precondition" is `wTop`, the unit of $\sqcap$, not `WEps sone`, which is the unit of $\cdot$ and asserts that the preceding trace is *empty*. Likewise `wFinally` must carry a trailing $\Sigma^*$, as `finally` does; without it the obligation reads "the trace ends with this event" and a task resolved anywhere but last is reported unmet.

A submitted task must be *resolved*, and there are two ways to do it. The future condition is accordingly a weighted disjunction — the quantitative counterpart of the `beginTx` obligation of §9.2 — with $\oplus$ = min selecting the cheaper route still open:

```
-- fut of submit: resolve by completing (1 step) or aborting (2 steps)
WAdd (wFinally (Tropical 1) (Atom "complete" (List [Num taskId])))
     (wFinally (Tropical 2) (Atom "abort"    (List [Num taskId])))

submitAndComplete = do { submit 1; complete 1 }
submitAndAbort    = do { submit 1; abort 1 }
twoTasks          = do { submit 1; complete 1; submit 2; complete 2 }

submitOnly        = submit 1          -- never resolved
```

| Program | $v(\mathit{fut})$, the minimum cost to discharge |
|---|---|
| `submitAndComplete` | 2 steps |
| `submitAndAbort` | 4 steps |
| `twoTasks` | 8 steps |
| `submitOnly` | $\infty$ |

Reading a residual here differs from the RE case, and this is the point of the instance. There is no syntactic $\neg\varnothing$ to compare against; one applies $v$ — `wNullable`, the weight the residual assigns to the empty continuation — and reads the number. A finite weight says the obligations are dischargeable and at what cost; $\infty$ is the semiring zero in the tropical case and says no trace discharges them at all. `submitOnly` is the only program of the four that is wrong, and it is the only one reporting $\infty$.

The costs of the correct programs are informative rather than merely non-infinite. `submitAndComplete` costs 2 and `submitAndAbort` costs 4: both resolve the task, and the difference is exactly the extra weight the tropical semiring assigns to the more expensive route, accumulated by $\otimes = +$ along the path. Under the probability semiring the same machinery reads differently again: the memory example of the accompanying artefact reports a residual weight of 94% for a matched `malloc`/`free` pair and 0% when the `free` is missing, so the residual quantifies confidence rather than cost. A Boolean verdict cannot express either reading.

### 9.5 Heap Memory (SL Instance)

Heap ownership is tracked via separation-logic predicates. Each `alloc` produces a **`Cell`**, and `free` consumes it.

```
alloc :: Addr → Val → Pledge IO SL ()
alloc addr val = Pledge $ return ((), Top, Cell addr val, Top)

free :: Addr → Val → Pledge IO SL ()
free addr val = Pledge $ return ((), Cell addr val, Emp, Top)

writeCell :: Addr → Val → Val → Pledge IO SL ()
writeCell addr old new = Pledge $ return ((), Cell addr old, Cell addr new, Top)

allocTwo  = do { alloc 0 1; alloc 1 2 }           -- two disjoint cells
allocFree = do { alloc 0 42; free 0 42 }          -- allocate, then release
leakOne   = do { alloc 0 1; alloc 1 2; free 0 1 } -- release only one
```

`allocTwo` behaves as intended: the composed *post* is `Cell 0 1` $*$ `Cell 1 2`, two disjointly owned cells joined by separating conjunction, which is exactly the reading Table 5 promises. The other two do not, and the reason is worth stating plainly rather than presenting a table that suggests otherwise.

`allocFree` yields *post* = `Cell 0 42`, not **emp**. Sequential composition is $*$, so the postcondition of a bind is $post(p) * post(q)$; `free` posts **emp**, and `Cell 0 42` $*$ **emp** = `Cell 0 42`. A release therefore contributes nothing. Separating conjunction *accumulates* footprint and has no way to retract it, so within this instance an operation cannot express that it gives ownership back. The consequences compound: `leakOne` produces the same *post* as `allocTwo`, making the leak invisible, and `alloc 0 0 ; write 0 7 ; free 0 7` produces `Cell 0 0` $*$ `Cell 0 7`, which asserts ownership of address 0 twice and is unsatisfiable in any heap.

The *fut* field is $\top$ throughout, for a related reason: the instance provides no test that would discharge an ownership obligation, so there is nothing for a non-trivial future condition to be checked against. What §8 establishes, then, is that the six operations of **`Composable`** have natural separation-logic readings and that the types line up — $*$ for $\cdot$, **emp** for $\varepsilon$, $-\!\!*$ for the quotient. What it does not establish is that those readings compose into a usable ownership discipline. Doing so needs a composition that is frame-aware, subtracting the footprint an operation consumes rather than only adding what it produces, together with a solver to discharge the resulting $-\!\!*$ obligations (§11). We report SL as evidence that the algebra generalises beyond traces, and not as a working instance; the three instances of §5–§7 are the ones that discharge obligations.

## 10 Related Work

*Future conditions and temporal verification.* The proposal this paper implements is due to Song et al. [2025], who introduce future conditions as a first-class extension to pre/post contracts at the level of a formal system. Song et al. [2024] apply a related RE encoding of temporal properties in ProveNFix to guide automated program repair. Pledge's contribution is a concrete, runnable library realisation: a monad transformer, a derivative-based discharge mechanism, and four **`Composable`** instances that neither prior paper instantiates.

*Pre/post-condition contracts in Haskell.* Liquid Haskell [Vazou et al. 2014] expresses pre/post-conditions as refinement types verified by an SMT solver. Findler and Felleisen [Findler and Felleisen 2002] establish runtime contract discipline for higher-order functions. Pledge complements these by adding a temporal future condition without requiring type-system extensions.

*Specifications as monads.* The idea that a specification can itself be monadic structure is the closest conceptual neighbour to this work. Hoare Type Theory [Nanevski et al. 2008] indexes a monad by a pre/post pair, making Hoare triples types. The *Dijkstra monad* [Swamy et al. 2013] instead indexes computations by predicate transformers, so that the weakest precondition of a composite is computed by bind; Ahman et al. [2017] derive such monads mechanically from an effect's definition, and Maillard et al. [2019] give the general account in which a Dijkstra monad is a monad morphism into a specification monad. Swierstra and Baanen [2019] develop the predicate-transformer view for effects in a dependently typed setting.

The shared idea is that specifications compose the way programs do; the difference is what composition computes. A Dijkstra monad propagates *backwards*: bind computes the weakest precondition of a sequence from its continuation's, and the result is a predicate transformer to be discharged by a verifier. **`Pledge`** propagates *forwards* as well, and its *fut* field has no counterpart in that line of work — a predicate transformer relates two states, whereas a future condition constrains the trace still to come, which no pre/post pair over states can express. The second difference is one of ambition. Dijkstra monads target full functional correctness in a dependently typed host and rely on a solver; **`Pledge`** targets temporal obligations in plain Haskell, computes a residual rather than a verification condition, and is correspondingly weaker and cheaper. Reading Equations (1)–(3) as a specification monad in the sense of Maillard et al. [2019], and asking what $\backslash$ is in that framework, is a question we leave open.

*Parameterised and graded monads.* Atkey [2009] introduces parameterised monads for Hoare-style state tracking; Katsumata [2014] generalises this to graded monads; Orchard and Yoshida [2016] shows effect systems and session types are instances. Pledge carries obligations as value-level annotation fields rather than indexing by effect type at the type level, preserving compatibility with the standard `Monad` class and **`do`**-notation.

*Resource management in Haskell.* The `bracket` combinator lexically scopes cleanup. `ResourceT` [Snoyman 2011], `Acquire`, and `Managed` thread finaliser queues through every bind. All three handle the cleanup half of a resource obligation but cannot express protocol or quantitative obligations, which require explicit future-condition propagation across bind steps.

*Linear types.* Linear Haskell [Bernardy et al. 2018] enforces "consume exactly once". Linear types enforce structural obligations (use at most once), whereas future conditions enforce temporal obligations (use in a specified order, or with a specified weight). A hybrid combining both could achieve stronger guarantees than either alone.

*Session types.* Session types [Gay and Hole 2005; Honda 1993] enforce communication protocols at the type level. The `sessiontypes` library [van Walree 2017] realises this via an indexed monad. Pledge is more lightweight: it operates within the standard `Monad` class, admits quantitative and heap-ownership obligations that session types do not, and is applicable beyond message-passing.

*Typestate.* Typestate [Strom and Yemini 1986] attaches a protocol state to a variable's type and checks transitions statically; Bierhoff and Aldrich [2007] extend this to aliased objects with access permissions. The obligations are the same ones **`Pledge`** targets — an opened file must be closed, a lock released — but the mechanism differs in where the state lives. Typestate puts it in the type, so the checker rejects a bad program and the protocol must be fixed before the code is written; **`Pledge`** puts it in a value, so a bad program compiles and its residual reports what it owes.

*Runtime verification.* Runtime verification [Leucker and Schallhart 2009] checks temporal properties against a concrete execution rather than a model, and is the setting **`Pledge`** most resembles. Havelund and Roşu [2001] monitor temporal formulae by rewriting them as events arrive — the residual formula after each event being precisely the obligation that remains, which is the same move the quotient makes here. MOP [Chen and Roşu 2007] generates efficient monitors from a specification attached to program points, and Copilot [Pike et al. 2010] generates hard-real-time C monitors from a Haskell EDSL.

Two things distinguish the present work. First, most monitors are automaton- or rewriting-based and report a Boolean verdict; here the monitor state *is* the specification, in the same algebra the programmer wrote, so a failure names the outstanding obligation rather than reporting that a property was violated at some step. Second, monitor composition is usually external — monitors are attached to a program, and combining them is a matter of running several at once. Here composition is the program's own $\gg\!=$, and the algebraic laws of §4 are about how obligations combine. The cost is that **`Pledge`** observes only what the annotations report: it computes the residual for the path actually taken (§3.4) but does not intervene, so a divergence between an annotation and the handler it shadows goes unnoticed. Closing that gap is the main practical work remaining (§11).

*Derivatives of regular expressions.* Brzozowski derivatives [Brzozowski 1964] have found application in parsing [Might et al. 2011; Owens et al. 2009] and model checking; Antimirov [1996] refine them into partial derivatives, whose sets stay smaller and which the quotient of §5 uses on the dividend. Equation (4) extends the construction to complement. What is new here is not the derivative but the use of a *language* quotient — of one specification by another, rather than by a single event — as the propagation rule of a monadic bind.

## 11 Discussion and Future Work

*Termination of the quotient.* The quotient of §5 is solved by a worklist traversal over the reachable pairs of derivatives, and terminates because both components range over finite sets: Brzozowski derivatives modulo associativity, commutativity and idempotence of $\vee$ and $\wedge$, and Antimirov partial derivatives outright. Cycle detection must therefore compare states modulo those identities; comparing them structurally is not enough, and a divisor whose derivative does not shrink — $\Sigma^*$, or any starred language — makes a naive recursion diverge.

The weighted instance is genuinely weaker here, and the reason is instructive. Modulo-ACI comparison is *unsound* over an arbitrary semiring, because neither $\oplus$ nor $\otimes$ need be idempotent: in the probability semiring $0.5 \oplus 0.5 = 1.0$. Worse, a weighted star has infinitely many distinct derivatives whenever its weight fails to stabilise under $\otimes$, as $([0.9]\ a)^*$ generates $0.9, 0.81, 0.729, \ldots$. Termination for WRE therefore holds for unit weights — which covers `wTop`, `wGlobally sone`, and the whole Boolean semiring, where WRE collapses to RE — and not in general. Postconditions in practice are unit-weight event sequences, so the restriction is not felt; identifying a semiring condition under which weighted derivatives are finite would remove it properly. Extending to context-free or $\omega$-regular specifications remains future work; `finally` here is a finite-trace operator, and genuine liveness over infinite runs is out of scope.

*Closing the gap between specification and real effects.* §3.4 pairs a **`Pledge`** term with a real `effectful` handler, so that one **`do`** block drives both and annotations may mention the values the handler produces. What that construction does not do is *guarantee* the two stay in step: the specification and the implementation are written side by side, and nothing prevents them drifting apart as the code evolves. The remedy we favour is to stop writing them separately. Rather than a **`Pledge`** term shadowing a real API, one can ship annotated wrappers — a **`Pledge`**-flavoured `Control.Concurrent.MVar`, `System.IO`, or resource API — in which each real operation already carries its own annotation. A programmer then writes ordinary code against a different import, and the specification is a byproduct of running the program rather than a parallel artefact to be maintained. This also answers the question of who writes annotations: the library author, once, rather than every caller.

*From reporting to enforcing.* As §1 noted, **`Pledge`** names an unmet obligation but does not prevent it: a program whose *pre* has collapsed to $\varnothing$ still runs to completion. Because the underlying monad is arbitrary, closing this is mechanical — a variant of bind that inspects the residual after each step and fails when it reaches $\varnothing$ turns the residual into a runtime guard, at the cost of a quotient computation per bind. The interesting question is not whether this is possible but what it should do on failure, and how much of the residual to recompute incrementally; we have not evaluated the cost.

*Making the side conditions unwriteable.* §4.5 and §4.6 identify two conditions an annotation must satisfy — postconditions denoting a single word, preconditions past-closed — and neither is expressed in the type `eff`. Both are easy to violate by writing the annotation that first comes to mind, and the second fails silently, reporting a contract violation for a correct program. Documentation is a poor defence. A better one is to withhold the raw constructors and expose only combinators that maintain the invariants: an `emits` smart constructor building a *post* from a list of events, and a `requires` constructor that wraps its argument as $\Sigma^* \cdot r$, so that a non-past-closed precondition cannot be written. This costs expressiveness — a primitive could no longer advertise a disjunctive postcondition — and buys a library whose users cannot accidentally leave the region where the monad laws hold. We think that is the right trade, and it is the main change we would make to the API.

*Toward solver-backed SL discharge.* §8 is explicit that the SL instance's **`leftQuotient`** records the magic wand symbolically. Wiring residual SL predicates into a decision procedure (Z3 via `sbv` or a dedicated SL prover such as Smallfoot) so that **`Wand`** nodes are checked for satisfiability, rather than merely recorded, is the concrete next step toward a soundness statement for SL analogous to Theorem 5.2.

*Parallel composition.* The **`Pledge`** monad is inherently sequential; concurrent programs require a parallel combinator $e_1 \parallel e_2$. In the RE instance this corresponds to the shuffle product $r_1 \sqcup\!\sqcup r_2$; the SL instance admits a parallel connective via the concurrent separation logic frame rule [O'Hearn 2007]. Extending **`Composable`** with a `parallel` operation is a natural next step.

*The event alphabet.* Unfolding a complement requires a working alphabet, and `firstWith` derives one from the operands as $\Sigma_0 = \mathit{atoms}(r_1) \cup \mathit{atoms}(r_2) \cup \{\texttt{Wildcard}\}$. The **`Wildcard`** element is what makes this open-world: it stands for "some event other than the named ones", which a specification can constrain without ever naming. Omitting it is not a conservative simplification but a soundness bug — a pair of operands naming no concrete event at all, such as $\Sigma^*$ or $\neg\varepsilon$, then has an empty working alphabet, no successor states to explore, and quotients to $\varnothing$ regardless of its actual denotation.

A real limitation remains one level down, in the event type rather than the alphabet. An `Event` is either **`Wildcard`**, matching everything, or an **`Atom`** matching a name *and* a payload exactly. There is no pattern between the two, so a specification cannot say "a `write` with any arguments" — the natural way to describe most protocols, and the first thing a user reaches for. Adding a name-indexed wildcard, or making the payload a matcher rather than a value, would remove the restriction without disturbing the algebra: only `subsumesEvent` and the derivative's **`Single`** clause depend on it.

## 12 Conclusion

We have presented *Pledge*, a Haskell library that realises Song et al. [2025]'s proposal of future conditions as a concrete monad transformer: a **`Pledge`** `m eff a` computation in underlying monad `m` that carries three annotation fields $(\mathit{pre}, \mathit{post}, \mathit{fut}) :: \mathit{eff}$, propagated algebraically through monadic bind via left- and right-quotient operations.

The central insight is that trace-based, arithmetic, quantitative, and heap-ownership obligations are structurally identical at the level of a six-operation **`Composable`** algebra. All four instances (RE, GRE, WRE, SL) share a single bind formula, and the diagnostic signal is uniform: a non-trivial residual in *pre* or *fut* identifies exactly which obligation remains unmet and why, a diagnostic we prove sound for the RE instance (Theorem 5.2). The monad laws for the $(\mathit{ret}, \mathit{post}, \mathit{fut})$ triple are verified mechanically in Lean 4 from twelve algebraic axioms.

Pledge requires no language extensions beyond standard type classes, composes via a standard `Monad` instance, and integrates with real effectful code through the monad transformer design: when `m = IO`, runtime values such as file handles or thread identifiers can be captured inside the underlying action and embedded directly into the annotation fields. Tightening the connection between specification and runtime execution — through effect-handler integration and solver-backed SL discharge — is the paper's main direction for future work.

## References

Danel Ahman, Cătălin Hriţcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy. 2017. Dijkstra Monads for Free. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 515–529. doi:10.1145/3009837.3009878

Valentin M. Antimirov. 1996. Partial Derivatives of Regular Expressions and Finite Automaton Constructions. *Theoretical Computer Science* 155, 2 (1996), 291–319. doi:10.1016/0304-3975(95)00182-4

Robert Atkey. 2009. Parameterised Notions of Computation. *Journal of Functional Programming* 19, 3–4 (2009), 335–376. doi:10.1017/S095679680900728X

Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: Practical Linearity in a Higher-Order Polymorphic Language. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 5 (2018), 29 pages. doi:10.1145/3158093

Kevin Bierhoff and Jonathan Aldrich. 2007. Modular Typestate Checking of Aliased Objects. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA)*. 301–320. doi:10.1145/1297027.1297050

Janusz A. Brzozowski. 1964. Derivatives of Regular Expressions. *J. ACM* 11, 4 (1964), 481–494. doi:10.1145/321239.321249

Feng Chen and Grigore Roşu. 2007. MOP: An Efficient and Generic Runtime Verification Framework. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. 569–588. doi:10.1145/1297027.1297069

Giuseppe De Giacomo and Moshe Y. Vardi. 2013. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*. 854–860.

Robert Bruce Findler and Matthias Felleisen. 2002. Contracts for Higher-Order Functions. In *Proceedings of the 7th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 48–59. doi:10.1145/581478.581484

Simon J. Gay and Malcolm Hole. 2005. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42, 2-3 (2005), 191–225. doi:10.1007/s00236-005-0177-z

Klaus Havelund and Grigore Roşu. 2001. Monitoring Programs Using Rewriting. In *Proceedings of the 16th IEEE International Conference on Automated Software Engineering (ASE)*. 135–143. doi:10.1109/ASE.2001.989799

Kohei Honda. 1993. Types for Dyadic Interaction. In *Proceedings of the 4th International Conference on Concurrency Theory (CONCUR) (Lecture Notes in Computer Science, Vol. 715)*. 509–523. doi:10.1007/3-540-57208-2_35

Shin-ya Katsumata. 2014. Parametric Effect Monads and Semantics of Effect Systems. *ACM SIGPLAN Notices* 49, 1 (2014), 633–646. doi:10.1145/2578855.2535846

Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303. doi:10.1016/j.jlap.2008.08.004

Kenji Maillard, Danel Ahman, Robert Atkey, Guido Martínez, Cătălin Hriţcu, Exequiel Rivas, and Éric Tanter. 2019. Dijkstra Monads for All. *Proceedings of the ACM on Programming Languages* 3, ICFP, Article 104 (2019). doi:10.1145/3341708

Matthew Might, David Darais, and Daniel Spiewak. 2011. Parsing with Derivatives: A Functional Pearl. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 189–195. doi:10.1145/2034773.2034801

Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. 2008. Hoare Type Theory, Polymorphism and Separation. *Journal of Functional Programming* 18, 5-6 (2008), 865–911. doi:10.1017/S0956796808006953

Peter W. O'Hearn. 2007. Resources, Concurrency, and Local Reasoning. *Theoretical Computer Science* 375, 1–3 (2007), 271–307. doi:10.1016/j.tcs.2006.12.035

Dominic Orchard and Nobuko Yoshida. 2016. Effects as Sessions, Sessions as Effects. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 568–581. doi:10.1145/2837614.2837634

Scott Owens, John Reppy, and Aaron Turon. 2009. Regular-expression Derivatives Re-examined. *Journal of Functional Programming* 19, 2 (2009), 173–190. doi:10.1017/S0956796808007090

Lee Pike, Alwyn Goodloe, Robin Morisset, and Sebastian Niller. 2010. Copilot: A Hard Real-Time Runtime Monitor. In *Runtime Verification — First International Conference (RV) (Lecture Notes in Computer Science, Vol. 6418)*. 345–359. doi:10.1007/978-3-642-16612-9_26

Michael Snoyman. 2011. conduit: Streaming Data Processing. https://hackage.haskell.org/package/conduit. Haskell package on Hackage; introduces the `ResourceT` transformer.

Yahui Song, Darius Foo, and Wei-Ngan Chin. 2025. Specifying and Verifying Future Conditions. In *Static Analysis — 32nd International Symposium, SAS 2025 (Lecture Notes in Computer Science, Vol. 16100)*. Springer, 90–112. doi:10.1007/978-3-032-07106-4_5

Yahui Song, Xiang Gao, Wenhua Li, Wei-Ngan Chin, and Abhik Roychoudhury. 2024. ProveNFix: Temporal Property-Guided Program Repair. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 11 (2024), 23 pages. doi:10.1145/3643737

Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* 12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929

Nikhil Swamy, Joel Weinberger, Cole Schlesinger, Juan Chen, and Benjamin Livshits. 2013. Verifying Higher-Order Programs with the Dijkstra Monad. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 387–398. doi:10.1145/2491956.2491978

Wouter Swierstra and Tim Baanen. 2019. A Predicate Transformer Semantics for Effects (Functional Pearl). *Proceedings of the ACM on Programming Languages* 3, ICFP, Article 103 (2019), 26 pages. doi:10.1145/3341707

The Mathlib Community. 2024. Mathlib4: The Math Library for Lean 4. https://leanprover-community.github.io/mathlib4_docs/.

Ferdinand van Walree. 2017. sessiontypes: Session Types for Haskell. https://hackage.haskell.org/package/sessiontypes.
Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. 2014. Refinement Types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 269–282. doi:10.1145/2628136.2628161